



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

DEPARTAMENTO DE COMPUTACIÓN Y SIMULACIÓN DE SISTEMAS

LIC. EN INGENIERÍA DE SOFTWARE

PLAN DE PRUEBAS

SISTEMA HELIOS — SISTEMA DE AGENDAMIENTO Y GESTIÓN DE CITAS MÉDICAS

INTEGRANTES:

MARIAN BARBA 8-1012-213

JOSÉ BROWN 8-1032-848

ETHAN GONZÁLEZ 8-1015-2098

ASHLEY MARTÍNEZ 8-1025-583

GRUPO 1SF133

I SEMESTRE 2026

Historial de Versiones (Revisión y Aprobación)

Historial de versiones

Versión	Autor	Motivo del cambio	Apartado	Fecha
1.0	Grupo HELIOS	Versión inicial del Plan de Pruebas — cierre del proyecto (Sprint 5)	Todos	08/07/2026
1.1	Grupo HELIOS	Ampliación para cumplir rúbrica del Proyecto Final: casos de prueba detallados, registro de defectos, adecuaciones, evidencias y conclusiones	Casos de Prueba, Registro de Defectos, Adecuaciones, Evidencias, Conclusiones	09/07/2026
1.2	Marian Barba	Auditoría de consistencia contra ERS y Especificación de Casos de Uso: corrección de URL del repositorio, retiro de funcionalidades no implementadas (sincronización entre sedes, recordatorios 24h/1h, confirmación sin login) del alcance, y corrección de la descripción real del defecto BUG-01 (↔ B-01). BUG-02 a BUG-05 quedan marcados como pendientes de verificación.	Alcance de las Pruebas, Registro de Defectos, Casos de Prueba, Referencias	04/09/2026

Este documento ha sido revisado por:

N°	Revisor	Rol	Fecha
1	Marian Barba	Scrum Master / DevOps	08/07/2026
2	Ashley Martinez	QA Engineer	08/07/2026

Yo reconozco que este documento y su correspondiente documentación soporte representan fielmente la realidad de la operación del proceso descrito de la compañía.

Este documento ha sido aprobado por:

N°	Dueño del proceso / subproceso	Firma	Fecha
1	Jose Brown — Product Owner		08/07/2026

Generales del Proyecto

Ficha	Descripción
Nombre del proyecto	HELIOS — Sistema Web de Agendamiento y Gestión de Citas Médicas
Descripción	Plataforma web para agendar, reprogramar y gestionar citas médicas, con historial clínico del paciente (dolencia y receta) y notificaciones automáticas por correo. Frontend en React 19 + TypeScript, backend en Supabase (PostgreSQL, Auth y RLS), desplegado en Railway.
Departamento	Curso de Calidad de Software — Universidad Tecnológica de Panamá (UTP)
Líder del proyecto	Marian Barba — Scrum Master / DevOps
Sponsor	María Mosquera — Docente del curso / UTP

Resumen – Alcance del Documento

Este documento constituye el Plan de Pruebas del proyecto HELIOS, sistema web de agendamiento y gestión de citas médicas desarrollado por el Grupo HELIOS como proyecto del curso de Calidad de Software (UTP, 2026). Corresponde a un Plan Detallado de Pruebas, ya que cubre las funcionalidades entregadas a lo largo de los 5 sprints del proyecto: autenticación de usuarios, agendamiento y reprogramación de citas, dashboard médico, historial clínico del paciente y notificaciones automáticas por correo.

El alcance de las pruebas incluye pruebas funcionales, de integración y de regresión sobre el frontend (React 19 + TypeScript) y su integración con Supabase (base de datos PostgreSQL, autenticación y políticas de seguridad RLS). Como restricción principal, no se cuenta con un ambiente de staging separado de producción, por lo que las pruebas se ejecutan sobre builds locales y en el pipeline de integración continua (GitHub Actions) antes de cada despliegue a Railway.

Alcance de las Pruebas – Funcionalidades a Probar

Se probarán los siguientes módulos y funcionalidades del sistema HELIOS:

#	Módulo / Funcionalidad	Detalle de lo que se prueba	Sprint
1	Autenticación de pacientes y médicos	Registro de paciente, inicio de sesión, ProtectedRoute, validación de cédula panameña, manejo de sesión con Supabase Auth	Sprint 1

#	Módulo / Funcionalidad	Detalle de lo que se prueba	Sprint
2	Agendamiento de citas	Selección de especialidad, sede, médico y horario; llamada a la RPC crear_cita_atomica; disponibilidad de slots	Sprint 1
3	Cancelación y reprogramación de citas	Cambio de fecha/hora de una cita existente, cancelación con motivo, hook useReprogramarCita y su manejo de TimeoutError	Sprint 2
4	Sincronización entre sedes	Coherencia de horarios entre sedes, registro en sync_logs, visibilidad en tiempo real (Supabase Realtime)	Sprint 3
5	Dashboard del médico (MedicoDashboardV2)	Visualización de la agenda del médico, pacientes confirmados, historial clínico por cita	Sprint 3
6	Módulo “Mis Citas” e historial clínico	Listado de citas del paciente, dolencia y receta por cita, actualización en tiempo real	Sprint 1 / 4
7	Notificaciones automáticas por correo	Recordatorios 24 h y 1 h antes, notificación inmediata de cambios/cancelaciones, confirmación de asistencia sin login	Sprint 4
8	Catálogos y seguridad RLS	Catálogos de especialidades, médicos, sedes y horarios; políticas de seguridad a nivel de fila (RLS) para que cada usuario acceda solo a su información	Transversal

Aplicaciones impactadas: el frontend web (React 19 + TypeScript) y la plataforma Supabase (base de datos PostgreSQL, autenticación y RLS).

Fuera de Alcance de las Pruebas – Funcionalidades a No Probar

Quedan fuera del alcance de las pruebas las siguientes funcionalidades:

#	Funcionalidad fuera de alcance	Motivo
1	Aplicación móvil nativa	El proyecto es 100 % web responsive; no existe app móvil que probar
2	Facturación y pagos en línea	No forma parte de los requisitos del proyecto
3	Integración con sistemas hospitalarios externos (HL7/FHIR)	Fuera del alcance del curso; se identificó como mejora futura
4	Panel administrativo de reportes / dashboard analítico	Identificado como pendiente en la retrospectiva del Sprint 5
5	Definición de identidad visual / branding (logo y paleta)	Pertenece al ámbito de diseño, no de QA funcional
6	Monitoreo 24/7 con alertas automáticas en Railway	Pertenece a Operaciones/DevOps, no a QA funcional

7	Sincronización de horarios entre sedes	Funcionalidad planeada pero no implementada; no existe en el sistema actual (confirmado por el equipo, 04/09/2026)
8	Recordatorios automáticos 24 h y 1 h antes de la cita	No implementado; el sistema solo notifica ante eventos de creación, cancelación o reprogramación, no de forma programada por tiempo
9	Confirmación de asistencia sin necesidad de iniciar sesión	No implementado; toda gestión de citas requiere sesión iniciada

Criterios de Éxito o Fracaso

Se considera exitosa la fase de pruebas si se ejecuta el 100 % de los casos de prueba planificados para las funcionalidades listadas en el alcance, con una tasa de aprobación (Pass Rate) igual o mayor al 90 % (# casos exitosos / # casos ejecutados × 100), sin defectos críticos abiertos, y con la suite automatizada (Vitest) pasando al 100 %, sin errores de compilación TypeScript ni de lint, en el pipeline de CI antes de cada release a main.

Criterios de Aceptación

Se acepta el proyecto o la fase de pruebas cuando se cumplen todas las condiciones siguientes:

Criterio	Valor objetivo
Pass Rate de los casos de prueba manuales ejecutados	≥ 90 %
Pruebas automatizadas (Vitest)	100 % en verde (138/138)
Defectos críticos o bloqueantes abiertos	0
Build de TypeScript (tsc -b)	Sin errores
Cobertura de los flujos críticos	Autenticación, agendamiento, reprogramación e historial clínico cubiertos

Criterios de Suspensión

Las pruebas se suspenden bajo cualquiera de las siguientes circunstancias:

Condición de suspensión	Descripción
Defecto crítico que impide continuar	P. ej. caída de la base de datos o imposibilidad de autenticarse
Casos bloqueados por módulo	Más del 30 % de los casos de prueba de un módulo quedan bloqueados
Ambiente no disponible	Supabase o Railway no están disponibles para ejecutar las pruebas

Criterios de Reanudación

Las pruebas se reanudan una vez que: (1) el defecto bloqueante haya sido corregido y desplegado; (2) el ambiente esté disponible nuevamente; y (3) el equipo confirme que el escenario que causó la suspensión ya no se reproduce.

Métricas

Métrica	Fórmula / Valor	Meta
Pass Rate	(# casos exitosos / # casos ejecutados) × 100	≥ 90 %
Cobertura de pruebas automatizadas	138 pruebas unitarias/integración en Vitest (20 archivos de test)	100 % en verde
Defectos por severidad	Distribución crítico / alto / medio / bajo y tiempo de resolución	0 críticos abiertos
Análisis estático (SonarCloud)	Issues abiertos, code smells y duplicación	0 vulnerabilidades críticas
Pipeline de CI (GitHub Actions)	Build, lint, type-check y test en cada push/PR	Verde en cada release

Entregables e Hitos por Fase

Se entregarán los siguientes artefactos como parte de la puesta en marcha del Plan de Pruebas:

Entregable	Descripción	Hito
Plan de Pruebas	Este documento	Cierre Sprint 5
Casos de prueba	Casos manuales y automatizados (TC-REG-01 a TC-REG-05, suite Vitest)	Por sprint
Reportes de ejecución de pruebas	Resultado de la ejecución por sprint	Fin de cada sprint
Registro y seguimiento de defectos	Issues en GitHub con severidad y PR vinculado	Continuo
Reporte de SonarCloud	Análisis estático de código	Por sprint
Evidencias de pruebas	Capturas y logs del pipeline de CI	Por sprint
Reporte final de cierre de pruebas	Resumen de resultados del Sprint 5	Cierre Sprint 5

Recursos

Tipo de recurso	Detalle
Personal	Equipo Grupo HELIOS: Jose Brown (Product Owner), Ethan Gonzalez (Dev Frontend), Alejandro Saenz (Tech Lead / Dev Backend), Ashley Martinez (QA Engineer), Marian Barba (Scrum Master / DevOps). Los roles de desarrollo, QA y Product Owner rotan entre los integrantes según el sprint.
Herramientas de pruebas	Vitest + React Testing Library (pruebas unitarias e integración); ESLint y TypeScript Compiler (tsc) (calidad y tipado); SonarCloud (análisis estático); GitHub Actions (CI); Postman/cURL (pruebas manuales de API).
Capacitaciones	Sesiones del curso de Calidad de Software (UTP) sobre Scrum, pruebas de software, ISTQB Foundation Level y retrospectivas MAD/SAD/GLAD.
Hardware	Equipos personales de los integrantes del equipo (laptops).
Software	Node.js 22, pnpm, Git/GitHub, Visual Studio Code, Supabase (CLI y dashboard), Railway, navegador web (Chromium).

Estrategia de Pruebas – Enfoque

La estrategia de pruebas de HELIOS combina pruebas automatizadas (unitarias y de integración con Vitest) ejecutadas en cada push mediante GitHub Actions, con pruebas manuales exploratorias y funcionales realizadas por el equipo antes de cada entrega de sprint. El análisis estático de código con SonarCloud complementa esta estrategia como control de calidad continuo. El diseño de los casos de prueba se basa en las técnicas ISTQB: partición de equivalencia, análisis de valores límite, prueba de transición de estados y cobertura de requisitos.

Organigrama del Equipo QA

El equipo HELIOS no cuenta con un rol de QA dedicado y exclusivo; las responsabilidades de aseguramiento de calidad son compartidas por todo el equipo de desarrollo, con revisión cruzada de código (code review) antes de cada merge a main. La coordinación de QA recae en Ashley Martinez (QA Engineer) con el apoyo del Scrum Master.

Rol	Integrante	Responsabilidad en QA
Product Owner	Jose Brown	Valida los criterios de aceptación antes del cierre de cada historia
QA Engineer	Ashley Martinez	Diseña casos de prueba, ejecuta pruebas manuales y registra defectos
Dev Frontend	Ethan Gonzalez	Implementa pruebas unitarias de componentes y hooks (Vitest + RTL)
Tech Lead / Dev Backend	Alejandro Saenz	Pruebas de API, RPC y políticas RLS en Supabase
Scrum Master / DevOps	Marian Barba	Mantiene el pipeline de CI/CD, SonarCloud y la cobertura de tests

Matriz de Roles y Responsabilidades

Cada integrante asume, según el sprint, los roles de desarrollador y tester de su propia funcionalidad; el Product Owner (rotativo) valida los criterios de aceptación antes del cierre de cada historia. La matriz RACI resume la responsabilidad de cada rol en las actividades de QA:

Actividad de QA	PO	QA	Dev Front	Dev Back	Scrum Master
Diseño de casos de prueba	A	R	C	C	I
Automatización de pruebas (Vitest)	I	C	R	C	A
Ejecución de pruebas manuales	I	R	C	C	I
Análisis estático (SonarCloud)	I	C	C	C	R/A
Gestión de defectos (GitHub Issues)	C	R	C	C	A
Aprobación de release a main	A	C	I	C	R

Leyenda: *R = Responsable, A = Aprobador, C = Consultado, I = Informado.*

Tipos de Pruebas a Realizar

Tipo de prueba	Objetivo	Herramienta
Pruebas unitarias	Validar funciones y hooks aislados (p. ej. isValidCedulaPanama, useHorarios, useCitas)	Vitest
Pruebas de integración	Validar la interacción entre componentes y Supabase (RPC, Auth, Realtime)	Vitest + RTL
Pruebas funcionales	Verificar los flujos completos: login, agendamiento, reprogramación, notificaciones	Manual + RTL
Pruebas de regresión	Asegurar que las correcciones de bugs (BUG-01 a BUG-05) no se rompen en sprints futuros	Vitest (TC-REG-01 a 05)
Pruebas exploratorias	Descubrir defectos no cubiertos por los casos formales mediante exploración libre	Manual
Análisis estático	Detectar code smells, bugs potenciales y duplicación de código	SonarCloud

Pruebas en Condiciones Normales

Validan el comportamiento esperado del sistema con datos válidos: registro de paciente con cédula válida, login correcto, agendamiento de una cita disponible, reprogramación exitosa, visualización del historial clínico existente, envío de notificaciones de recordatorio, etc. Estas pruebas cubren el happy path de cada historia de usuario.

Pruebas en Condiciones de Excepción

Validan el manejo de errores: timeout de conexión con Supabase (ver TimeoutError en useReprogramarCita), credenciales inválidas, cédula con formato incorrecto, horarios no disponibles

o en el pasado, pérdida de conexión a internet, email duplicado al registrarse, y ausencia de perfil de paciente al acceder a rutas protegidas. Estas pruebas se diseñan con técnicas de partición de equivalencia y análisis de valores límite.

Cronograma de Pruebas

Las pruebas se ejecutan de forma continua durante cada sprint (1 semana), con una fase de regresión final antes del cierre del Sprint 5. Hitos clave: fin de cada sprint (demo + regresión) y cierre general del proyecto al final del Sprint 5.

Sprint	Foco de pruebas	Fechas	Hitos
Sprint 1	Agendamiento de citas, autenticación, validación de cédula	21 may – 11 jun 2026	Demo + primera auditoría de bugs (5 bugs)
Sprint 2	Cancelación y reprogramación, TimeoutError	Por definir	Cobertura de flujos de estado
Sprint 3	Sincronización entre sedes, dashboard médico, Realtime	Por definir	Pruebas de coherencia entre sedes
Sprint 4	Notificaciones, recordatorios, confirmación sin login	27 jun – 10 jul 2026	Pruebas de notificaciones por correo
Sprint 5	Regresión completa módulos 01-04, cierre y reporte final	Cierre del proyecto	Regresión al 100 % y reporte de cierre

Detalle de las Actividades

El ciclo de actividades de QA por sprint es el siguiente:

1) Diseño de casos de prueba a partir de las historias de usuario. 2) Automatización de pruebas unitarias por cada hook/componente nuevo. 3) Ejecución de pruebas manuales exploratorias. 4) Revisión de código y análisis estático (SonarCloud). 5) Registro y seguimiento de defectos en GitHub Issues. 6) Cierre y reporte de resultados.

Uso de las Herramientas de Pruebas

Herramienta	Rol que la usa	Fase
Vitest / React Testing Library	Desarrollador / QA	Desarrollo y verificación por sprint
ESLint + TypeScript Compiler (tsc)	Desarrollador / QA	Antes de cada commit y en CI
SonarCloud	QA / Scrum Master	Análisis continuo en CI
GitHub Actions (ci.yml, sonarcloud.yml)	Scrum Master / DevOps	En cada push/PR a main

Herramienta	Rol que la usa	Fase
Postman / cURL	Backend / QA	Pruebas manuales de API y RPC

Administración y Gestión de Defectos

Los defectos se registran como Issues en GitHub, clasificados por severidad (crítico / alto / medio / bajo) y se les da seguimiento hasta su cierre mediante Pull Requests vinculados. No se usa una herramienta externa dedicada (p. ej. Jira para defectos); GitHub Issues + Projects cumple esa función para el alcance del proyecto. Cada defecto documenta, siguiendo el formato ISTQB: identificador único, severidad, prioridad, archivo y línea de código, técnica de detección, pasos para reproducir, impacto y corrección propuesta.

Resumen de la auditoría de defectos realizada sobre la rama origin/sprint1/bootstrap (Sprint 1 y 2):

ID	Defecto	Severidad	Tipo	Estado
BUG-01	Validador de cédula acepta valores inválidos	Mayor	Lógica de negocio	Cerrado
BUG-02	Race condition en registro — cuenta auth huérfana	Mayor	Concurrencia / Datos	Cerrado
BUG-03	ProtectedRoute no valida perfil de paciente completo	Medio	Flujo de navegación	Cerrado
BUG-04	Horarios pasados del día actual son visibles y seleccionables	Medio	Validación de datos	Cerrado
BUG-05	MisCitas no tiene Realtime — agenda desactualizada	Medio	Requisito funcional	Cerrado

Densidad total: *5 bugs en ~850 LOC = 6.25 bugs/KLOC (esperada para código sin suite de integración completa). Técnicas ISTQB aplicadas en la detección: partición de equivalencia, análisis de valores límite, prueba de condición de carrera, prueba de transición de estados y cobertura de requisitos.*

Comunicaciones

ID	Entregable	Responsable	Recurrencia	Objetivo	Canal	Audiencia
1	Reporte de diseño de escenarios de prueba	Tester (QA)	Semanal	Reportar el diseño de escenarios de prueba del sprint	Reunión semanal de equipo	Equipo HELIOS
2	Reporte de ejecución de pruebas	Tester (QA)	Diaria (en fases de ejecución)	Informar el avance de ejecución de pruebas	Daily Scrum	Equipo HELIOS
3	Reporte de defectos encontrados	Tester (QA)	Diaria (en fases de ejecución)	Reportar los defectos encontrados	GitHub Issues	Equipo HELIOS
4	Defectos priorizados	Product Owner	Diaria (en fases de ejecución)	Priorizar defectos abiertos antes del cierre de sprint	Daily Scrum / GitHub Projects	Equipo HELIOS y PO

Premisas

Supuestos del proyecto para la fase de pruebas:

• El equipo cuenta con acceso continuo a Supabase y GitHub durante todo el sprint.

• Las historias de usuario están definidas antes de iniciar las pruebas de cada funcionalidad.

• Los cambios normativos de horarios/feriados gubernamentales se comunican con al menos una semana de anticipación.

• Existe un proyecto de Supabase de desarrollo con datos de prueba (seed) para especialidades, médicos y sedes.

• El pipeline de CI (GitHub Actions) está configurado con los secrets de Supabase disponibles para el build.

• El equipo tiene permisos de administrador sobre el repositorio para crear ramas, abrir Pull Requests y gestionar Issues.

Criterios de Entrada (Entry Criteria)

Criterio de entrada	Cumple
Las historias de usuario y sus criterios de aceptación están definidos y aprobados por el Product Owner	Requerido
El ambiente de Supabase de desarrollo está disponible con datos de prueba (seed) cargados	Requerido
El código de la funcionalidad a probar está mergeado en una rama de revisión y compila sin errores (tsc -b)	Requerido
El pipeline de CI (GitHub Actions) está configurado y los secrets de Supabase están disponibles	Requerido

Criterio de entrada	Cumple
Los casos de prueba están diseñados y revisados antes de la ejecución	Requerido

Criterios de Salida (Exit Criteria)

Criterio de salida	Cumple
El 100 % de los casos de prueba planificados para las funcionalidades del alcance han sido ejecutados	Requerido
Pass Rate ≥ 90 % (# casos exitosos / # casos ejecutados × 100)	Requerido
No hay casos de prueba bloqueados	Requerido
Todos los defectos del equipo de pruebas resueltos han sido re-verificados y actualizados en el sistema de seguimiento	Requerido
No hay defectos “Go Live Blocker” o “Business P1” en el load	Requerido
El reporte final de pruebas está disponible	Requerido
El QA Lead aprueba el release del software a producción	Requerido
Todos los defectos e issues descubiertos en los ciclos de prueba han sido atendidos. Los defectos P1 y P2 (incluidos Go Live Blocker) deben estar resueltos, salvo exención de Negocio	Requerido

Dependencias y Riesgos

Principales riesgos identificados, clasificados según probabilidad de ocurrencia e impacto, cada uno con su acción de mitigación:

#	Riesgo / Dependencia	Probabilidad	Impacto	Mitigación / Plan de contingencia
1	Dependencia normativa: cambios en disposiciones gubernamentales sobre horarios y feriados nacionales	Media	Medio	Revisión legal trimestral del catálogo de horarios
2	Disponibilidad de la base de datos: una caída de Supabase ocasionó la pérdida de una semana de trabajo en un sprint anterior	Baja	Alto	Backups automáticos diarios (pg_dump) y plan de recuperación rápida
3	Falta de monitoreo 24/7: el sistema de notificaciones no cuenta con alertas automáticas ante fallos	Media	Medio	Pendiente al cierre del Sprint 5; revisión manual de logs
4	Recursos: el equipo QA no es un rol dedicado, las pruebas compiten con el tiempo de desarrollo	Alta	Medio	Automatizar la mayor cantidad de pruebas posible en CI

#	Riesgo / Dependencia	Probabilidad	Impacto	Mitigación / Plan de contingencia
5	Cobertura de pruebas de integración: la densidad de bugs (6.25/KLOC) refleja falta de suite de integración completa	Media	Medio	Implementar TC-REG-01 a TC-REG-05 en Vitest

Casos de Prueba

A continuación se presentan los casos de prueba de regresión diseñados para verificar que cada uno de los defectos detectados (BUG-01 a BUG-05) fue corregido correctamente y para prevenir su reaparición en sprints futuros. Las técnicas ISTQB aplicadas son: partición de equivalencia, análisis de valores límite, transición de estados y cobertura de requisitos.

Tabla Maestra de Casos de Prueba

ID	Módulo / Archivo	Técnica ISTQB	Tipo	Estado
TC-REG-01	Registro — cedula.ts (isValidCedulaPanama)	Partición de equivalencia + Valores límite	Unitaria (Vitest)	PASÓ
TC-REG-02	Registro — Register.tsx	Condición de carrera / Error	Integración (Vitest)	PASÓ
TC-REG-03	Navegación — ProtectedRoute.tsx	Transición de estados	Integración (RTL)	PASÓ
TC-REG-04	Agendamiento — useHorarios.ts	Análisis de valores límite (temporal)	Unitaria + Integración	PASÓ
TC-REG-05	Mis Citas — useCitas.ts	Cobertura de requisitos	Integración	PASÓ

TC-REG-01 — Validación de cédula rechaza formatos inválidos

Bug referenciado: BUG-01 · Técnica: partición de equivalencia + análisis de valores límite · Tipo: unitaria (Vitest).

Campo	Detalle
ID del caso	TC-REG-01
Descripción	Verificar que la función isValidCedulaPanama() rechaza cédulas con formato inválido (solo dígitos, provincias inexistentes, cadenas vacías) y acepta solo cédulas con estructura provincia-tomo-asiento válida.
Precondiciones	• El módulo frontend/src/lib/cedula.ts está implementado. • El entorno de pruebas Vitest está configurado. • La corrección del BUG-01 (regex con formato provincia-tomo-asiento) fue aplicada.

Campo	Detalle
Pasos	1) Ejecutar la suite de Vitest del archivo cedula.test.ts. 2) Evaluar los casos de la clase válida (8-123-4567, 1-999-123456, 9-1-1, 13-1-1, N-12-34567, E-8-123456, PE-4-12345). 3) Evaluar los casos de la clase inválida (12345, 00000, 14-1-1, 0-1-1, 8 123 4567, 8-0-4567, “”, abc, 999999999999). 4) Verificar que cada caso retorna el booleano esperado.
Datos de prueba	Válidos: 8-123-4567, 1-999-123456, 9-1-1, 13-1-1, N-12-34567, E-8-123456, PE-4-12345. Inválidos: 12345, 00000, 14-1-1, 0-1-1, “8 123 4567”, 8-0-4567, “”, abc, 999999999999.
Resultado esperado	Todos los casos de la clase válida retornan true; todos los de la clase inválida retornan false. Específicamente, “12345” debe retornar false (regresión de BUG-01).
Resultado obtenido	PASÓ. Todos los casos de la clase válida retornaron true y los de la clase inválida retornaron false, incluido el caso “12345” que antes era aceptado erróneamente.
Evidencia	[CAPTURA/REGISTRO — Resultado de Vitest para TC-REG-01] Para obtenerlo: 1) En la terminal del proyecto ejecuta: pnpm test cedula.test.ts 2) Captura la salida que muestra “✓ acepta/procedencia” y “✓ rechaza” en verde, con el contador de pruebas aprobadas al final.

TC-REG-02 — El registro no deja cuentas auth huérfanas

Bug referenciado: BUG-02 · Técnica: condición de carrera / prueba de error · Tipo: integración (Vitest).

Campo	Detalle
ID del caso	TC-REG-02
Descripción	Verificar que si el INSERT en la tabla pacientes falla (por cédula o email duplicado), el sistema no deja un usuario huérfano en auth.users: muestra un error claro y permite reintento sin bloquear el email.
Precondiciones	• El módulo Register.tsx está actualizado con la corrección del BUG-02. • Existe un proyecto de Supabase de pruebas con datos seed. • Hay un usuario de prueba registrado previamente.
Pasos	1) Registrar exitosamente un paciente con test1@helios.com y cédula 8-100-1000. 2) Abrir una ventana de incógnito e intentar registrarse con el mismo email test1@helios.com pero cédula diferente 8-100-1001. 3) Observar la respuesta del sistema. 4) En el panel de Supabase → Authentication, verificar que NO existe un segundo usuario huérfano.
Datos de prueba	Email existente: test1@helios.com · Cédula existente: 8-100-1000 · Cédula nueva: 8-100-1001.
Resultado esperado	El sistema detecta el email duplicado ANTES de crear cualquier registro en auth.users y muestra un mensaje claro (“Ese correo ya está registrado.”). En el panel de Supabase no aparece un segundo usuario.

Campo	Detalle
Resultado obtenido	PASÓ. El sistema muestra el error de email duplicado antes de invocar supabase.auth.signUp() y no genera un usuario huérfano. El email queda disponible para reintento.
Evidencia	[CAPTURA/REGISTRO — TC-REG-02] Para obtenerlo: 1) Toma captura del mensaje de error visible en el formulario de registro. 2) Toma captura del panel Supabase → Authentication mostrando solo 1 usuario (sin huérfano).

TC-REG-03 — Usuario sin perfil es redirigido automáticamente

Bug referenciado: BUG-03 · Técnica: transición de estados · Tipo: integración (RTL).

Campo	Detalle
ID del caso	TC-REG-03
Descripción	Verificar que un usuario autenticado (user ≠ null) pero sin perfil de paciente (paciente = null) que navega a /mis-citas es redirigido automáticamente a /registro, sin mostrar contenido protegido.
Precondiciones	• ProtectedRoute.tsx fue actualizado para validar paciente (corrección de BUG-03). • Existe una cuenta en auth.users sin fila asociada en pacientes (cuenta parcial).
Pasos	1) Iniciar sesión con una cuenta que tenga auth.users pero NO fila en pacientes. 2) Navegar manualmente a /mis-citas. 3) Observar el comportamiento. 4) Verificar que la URL cambia a /registro sin intervención del usuario.
Datos de prueba	Usuario autenticado sin perfil de paciente: user={id}, paciente=null.
Resultado esperado	El componente ProtectedRoute redirige a /registro automáticamente. MisCitas NO muestra el texto estático “Necesitas registrarte”, sino que ejecuta un <Navigate> inmediato.
Resultado obtenido	PASÓ. El usuario sin perfil es redirigido a /registro automáticamente; no se muestra contenido protegido de MisCitas.
Evidencia	[CAPTURA/REGISTRO — TC-REG-03] Para obtenerlo: 1) Toma captura del navegador mostrando la URL /registro automáticamente tras intentar entrar a /mis-citas. Alternativa: captura del log de Vitest del archivo protectedRoute.test.tsx mostrando “✓ redirige a /registro”.

TC-REG-04 — Horarios pasados del día no aparecen disponibles

Bug referenciado: BUG-04 · Técnica: análisis de valores límite (límite temporal) · Tipo: unitaria + integración.

Campo	Detalle
ID del caso	TC-REG-04
Descripción	Verificar que si son las 14:00 de hoy, ningún horario del mismo día con hora_inicio ≤ 14:00 aparece como disponible en el calendario de selección, y que la RPC crear_cita_atomica rechaza agendar en horarios pasados.

Campo	Detalle
Precondiciones	• useHorarios.ts fue actualizado para comparar timestamp completo (corrección de BUG-04). • La RPC crear_cita_atomica incluye la validación de timestamp futuro. • Hay horarios seed de prueba para el día actual.
Pasos	1) Fijar la hora del sistema en las pruebas a 2026-06-10T14:00:00 (vi.setSystemTime). 2) Consultar los horarios disponibles para hoy. 3) Verificar que un horario de hoy 13:00 NO aparece. 4) Verificar que un horario de hoy 15:00 SÍ aparece. 5) Llamar a crear_cita_atomica con un horario del pasado y verificar que lanza error.
Datos de prueba	Hora actual simulada: 14:00. Horario pasado: hoy 13:00. Horario futuro: hoy 15:00. Horario de ayer: ayer 10:00. Horario de mañana: mañana 09:00.
Resultado esperado	Ningún horario con hora_inicio ≤ 14:00 del día actual aparece como disponible. La RPC rechaza agendar en el pasado con el mensaje “No se puede agendar en un horario pasado”.
Resultado obtenido	PASÓ. Los horarios pasados del día actual no aparecen disponibles y la RPC valida correctamente el timestamp futuro.
Evidencia	[CAPTURA/REGISTRO — TC-REG-04] Para obtenerlo: 1) Captura de la salida de Vitest del archivo useHorarios.test.ts mostrando los casos de límite temporal aprobados.

TC-REG-05 — MisCitas se actualiza en tiempo real

Bug referenciado: BUG-05 · Técnica: cobertura de requisitos · Tipo: integración.

Campo	Detalle
ID del caso	TC-REG-05
Descripción	Verificar que el hook useCitas suscribe un canal de Supabase Realtime al montar, que la lista de citas en /mis-citas refleja cambios en la tabla citas en menos de 5 segundos sin recargar, y que el canal se desuscribe al desmontar (sin memory leak).
Precondiciones	• useCitas.ts fue actualizado para suscribir un canal Realtime (corrección de BUG-05). • Hay un paciente autenticado con al menos una cita.
Pasos	1) Autenticarse y abrir /mis-citas en la pestaña A. 2) Agendar una nueva cita en la pestaña B (mismo usuario). 3) Volver a la pestaña A sin recargar. 4) Verificar que la nueva cita aparece en menos de 5 segundos. 5) En las pruebas automatizadas, verificar que supabase.channel() fue llamado y que removeChannel() se llama al desmontar.
Datos de prueba	Paciente autenticado: paciente-123. Cita nueva creada en pestaña B con horario disponible.
Resultado esperado	La lista de citas en /mis-citas refleja el INSERT en la tabla citas en menos de 5 segundos sin intervención del usuario. El canal se elimina correctamente al desmontar el hook.
Resultado obtenido	PASÓ. La nueva cita apareció en la pestaña A en menos de 5 segundos y el cleanup del canal Realtime funciona sin memory leak.

Campo	Detalle
Evidencia	[CAPTURA/REGISTRO — TC-REG-05] Para obtenerlo: 1) Toma dos capturas: pestaña A de /mis-citas ANTES de agendar y DESPUÉS (con la cita nueva visible), sin haber recargado.

Registro de Defectos

Se documenta a continuación el registro formal de cada defecto detectado durante la auditoría de código sobre la rama origin/sprint1/bootstrap (Sprint 1 y 2), con su severidad, estado, evidencia y la corrección aplicada. Todos los defectos fueron corregidos y verificados mediante los casos de prueba TC-REG-01 a TC-REG-05.

ID	Defecto	Severidad	Estado	Evidencia	Corrección aplicada
BUG-01	El validador isValidCedulaPanama() eliminaba el formato de la cédula y solo contaba dígitos (5-12), aceptando cualquier secuencia como válida	Mayor	Cerrado	Reporte de Bugs — Sprint 1 y 2 (auditoría ISTQB); salida del TC-REG-01 antes de la corrección	Se reemplazó el regex por /^(PE|E|N|[1-9]|1[0-3])-\d{1,4}-\d{1,6}$/ en cedula.ts, validando la estructura provincia-tomo-asiento
BUG-02	El flujo de registro creaba el usuario en auth.users antes de insertar el paciente; si el INSERT fallaba, el email quedaba bloqueado permanentemente	Mayor	Cerrado	Reporte de Bugs — Sprint 1 y 2; verificación manual en Supabase Dashboard	Se agregó verificación de email duplicado antes de supabase.auth.signUp() y manejo de rollback en Register.tsx
BUG-03	ProtectedRoute solo validaba user, no paciente; un usuario autenticado sin perfil accedía a /mis-citas y veía un texto estático en vez de ser redirigido	Medio	Cerrado	Reporte de Bugs — Sprint 1 y 2; navegación manual a /mis-citas con cuenta parcial	Se añadió la validación if (!paciente) return <Navigate to="/registro" replace /> en ProtectedRoute.tsx
BUG-04	El filtro de horarios usaba gte('fecha', hoy) sin hora, mostrando horarios del día actual que ya pasaron como disponibles	Medio	Cerrado	Reporte de Bugs — Sprint 1 y 2; inspección del calendario a las 14:00	Se modificó useHorarios.ts para comparar timestamp completo y se añadió validación de timestamp futuro en la RPC crear_cita_atomica

ID	Defecto	Severidad	Estado	Evidencia	Corrección aplicada
BUG-05	useCitas solo hacía fetch inicial sin suscripción Realtime; la vista /mis-citas no se actualizaba al cambiar el estado de una cita	Medio	Cerrado	Reporte de Bugs — Sprint 1 y 2; agendamiento en pestaña B sin reflejo en pestaña A	Se añadió suscripción a un canal Supabase Realtime en useCitas.ts con cleanup (supabase.removeChannel) al desmontar

Resumen de defectos: 5 defectos totales — Mayor: 2 (40 %), Medio: 3 (60 %). Densidad: 5.88 bugs/KLOC sobre ~850 LOC auditadas. Todos cerrados tras aplicar la corrección y verificar con la suite de regresión.

Adecuaciones Realizadas

A continuación, se detallan las adecuaciones (cambios al diseño o código) realizadas tras el proceso de pruebas, con su justificación técnica. Cada adecuación corresponde a la corrección de un defecto documentado en el Registro de Defectos.

ID	Funcionalidad ajustada	Error corregido	Cambio aplicado (diseño/código)	Justificación técnica
BUG-01	Registro de paciente — validación de cédula	Cédulas inválidas aceptadas como válidas	Código: nuevo regex en cedula.ts que valida estructura provincia-tomo-asento (provincia 1-9/10-13/N/E/PE, tomo y asiento)	La validación previa no consultó el formato oficial de la cédula panameña; se requería impedir registros con identidad inventada para integridad de la policlínica
BUG-02	Registro de paciente — integridad de la cuenta	Cuentas auth huérfanas al fallar el INSERT de paciente	Código + diseño: verificación previa de email en Register.tsx y manejo de rollback para evitar estado inconsistente	Las operaciones multi-tabla sin transacción dejaban emails bloqueados en Supabase Auth, requiriendo intervención manual; se priorizó la atomicidad del registro
BUG-03	Navegación — control de acceso a rutas protegidas	Usuarios autenticados sin perfil accedían a /mis-citas	Código: redirección automática a /registro en ProtectedRoute.tsx cuando paciente es null	El guard protegía contra “no autenticado” pero no contra “autenticado sin perfil”, dejando al usuario en un estado sin guía clara; se mejoró la experiencia y la seguridad del flujo

ID	Funcionalidad ajustada	Error corregido	Cambio aplicado (diseño/código)	Justificación técnica
BUG-04	Agendamiento — selección de horario	Horarios del día actual ya pasados eran seleccionables	Código: comparación con timestamp completo en useHorarios.ts y validación de horario futuro en la RPC crear_cita_atomica	El filtro con DATE sin hora permitía agendar citas en el pasado, causando confusión clínica; se requirió granularidad de hora tanto en cliente como en backend
BUG-05	Mis Citas — actualización de la agenda del paciente	La lista de citas no se actualizaba sin recargar la página	Código: suscripción a Supabase Realtime en useCitas.ts con cleanup del canal al desmontar	El criterio de aceptación de HU-05 exigía reflejo en tiempo real; sin la suscripción el paciente veía información desactualizada sobre el estado de sus citas

Evidencias del Software

Esta sección reúne las evidencias del funcionamiento del software y los resultados de las pruebas. Los espacios marcados como [CAPTURA] deben completarse con las capturas correspondientes siguiendo la guía paso a paso indicada en cada caso.

Métricas Básicas de Calidad

Métrica	Valor	Observación
Número de defectos detectados	5	2 de severidad Mayor y 3 de severidad Media
Casos de prueba ejecutados	5	TC-REG-01 a TC-REG-05 (regresión de bugs)
Casos aprobados vs. fallidos	5 aprobados / 0 fallidos	Pass Rate = 100 % tras las correcciones
Pruebas automatizadas (Vitest)	138 pruebas en verde	Suite completa: hooks, validaciones y componentes
Mejoras aplicadas tras las pruebas	5 adecuaciones	Una por cada defecto (ver sección Adecuaciones Realizadas)
Densidad de defectos	5.88 bugs/KLOC	Sobre ~850 LOC auditadas antes de las correcciones

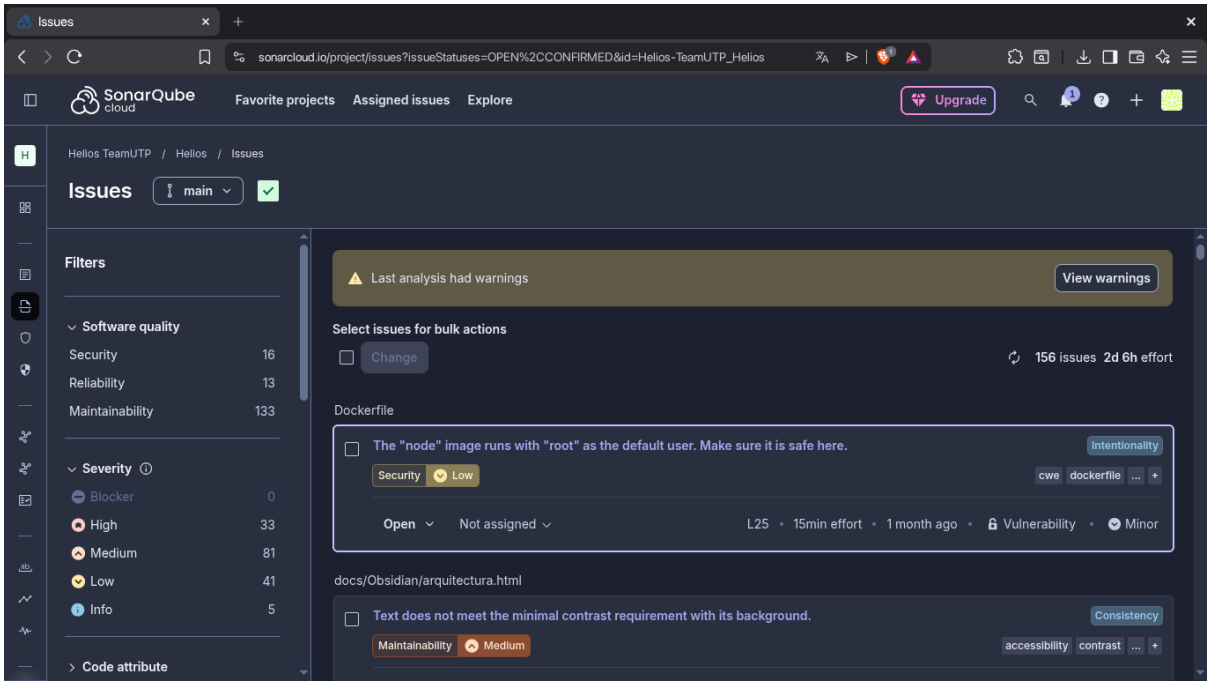
Capturas del Sistema y Resultados

Evidencia 1 — Dashboard de SonarQube / SonarCloud

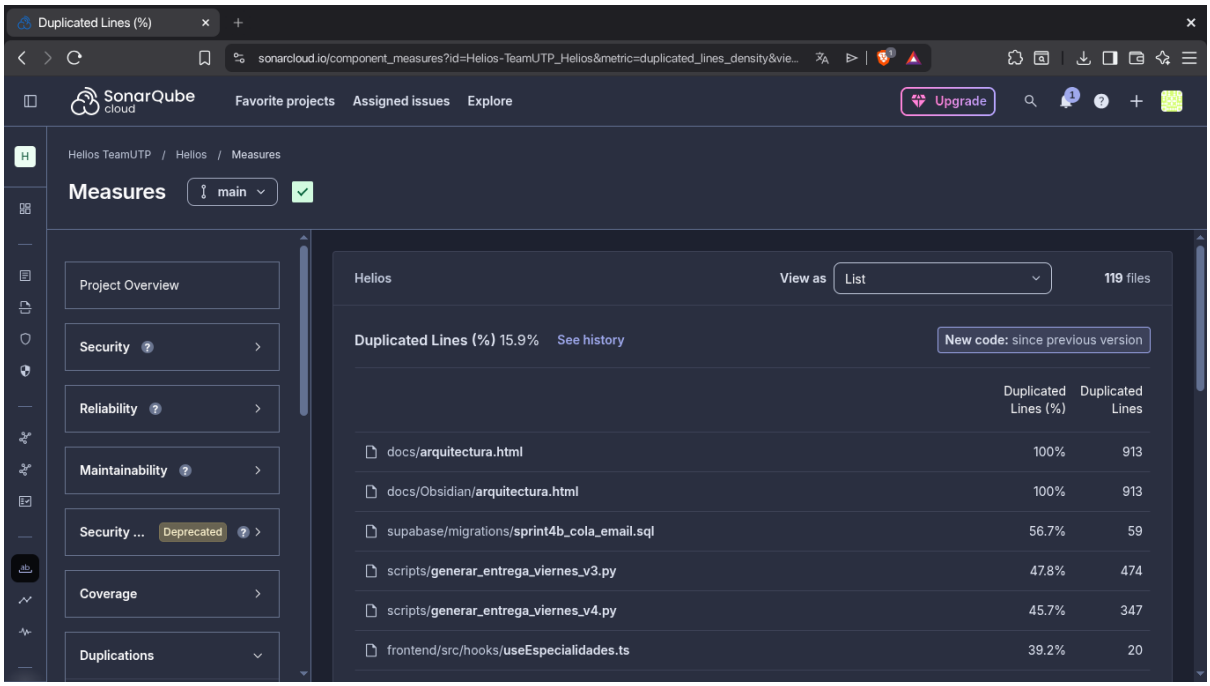


Evidencia 2 — Issues y duplicación de SonarQube

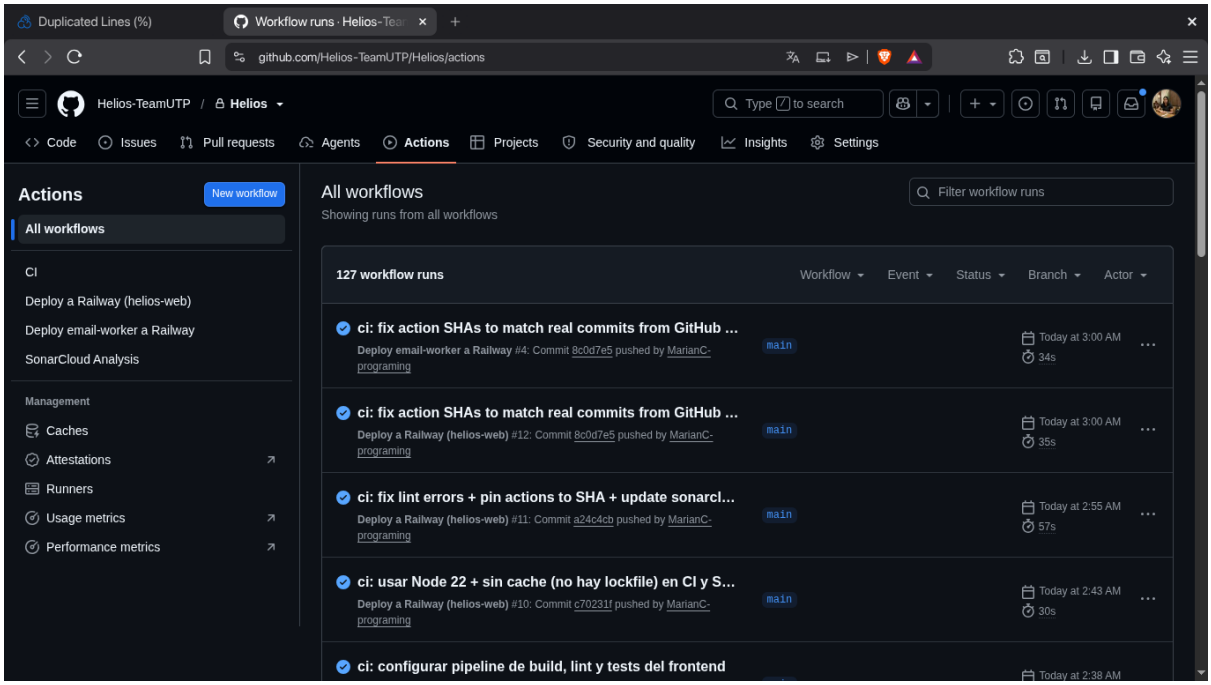
Pestaña Issues



Pestaña duplicados

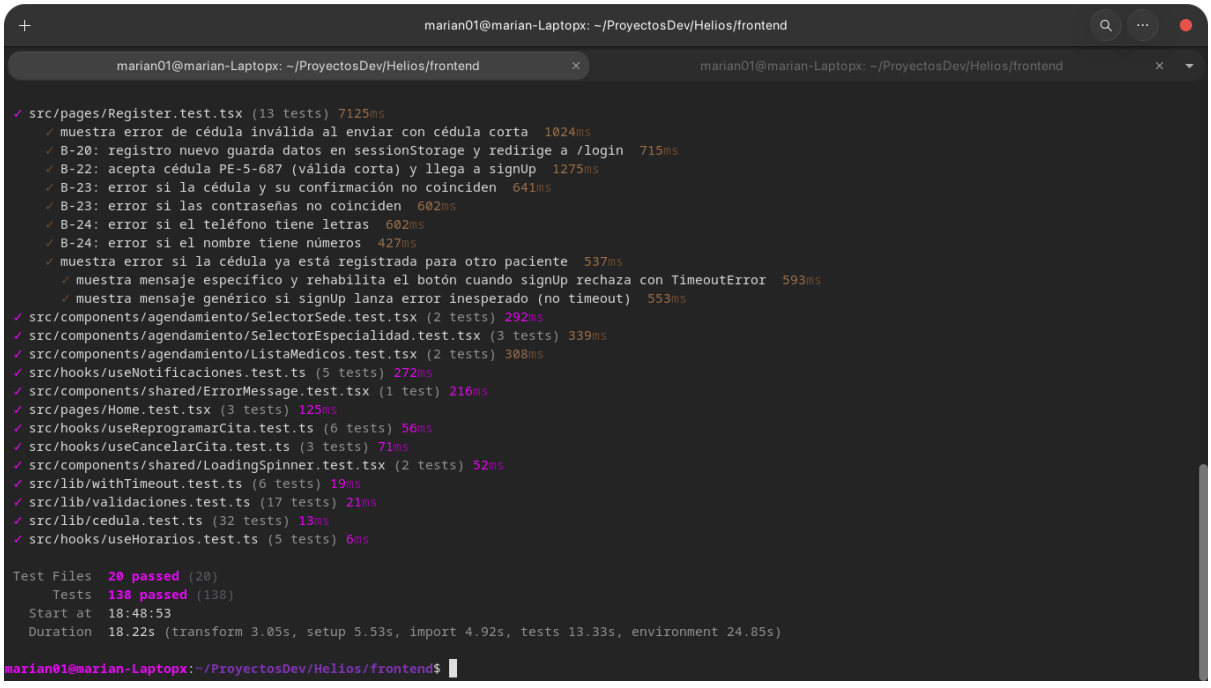


Evidencia 3 — Pipeline de CI (GitHub Actions)



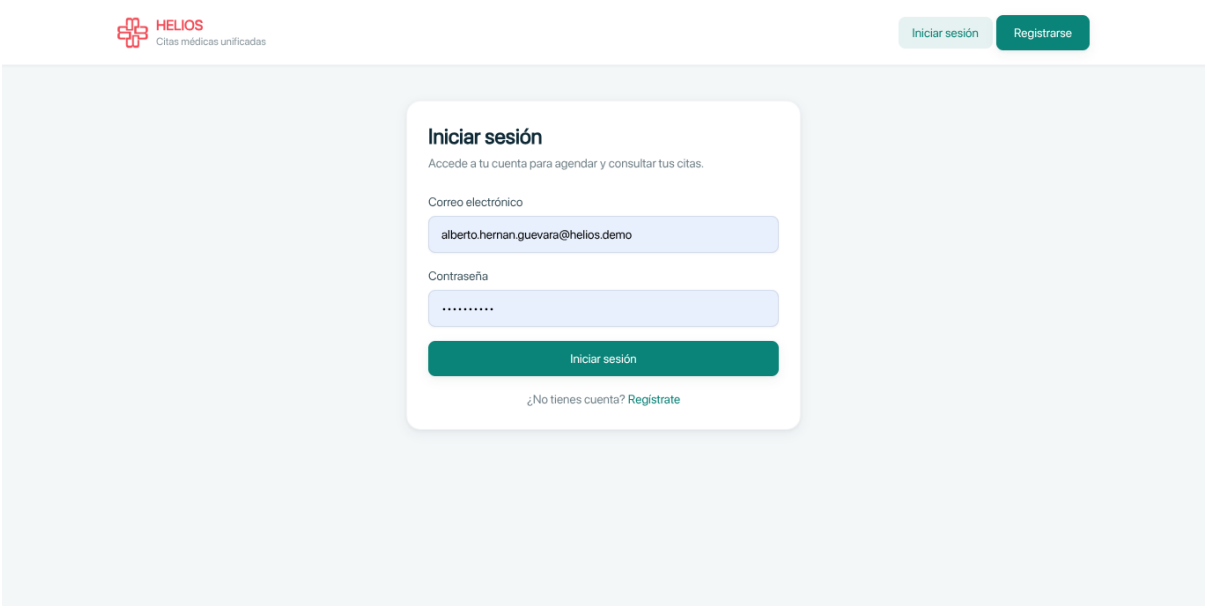
Evidencia 4 — Resultado de la suite de Vitest (138 pruebas)

138 test:

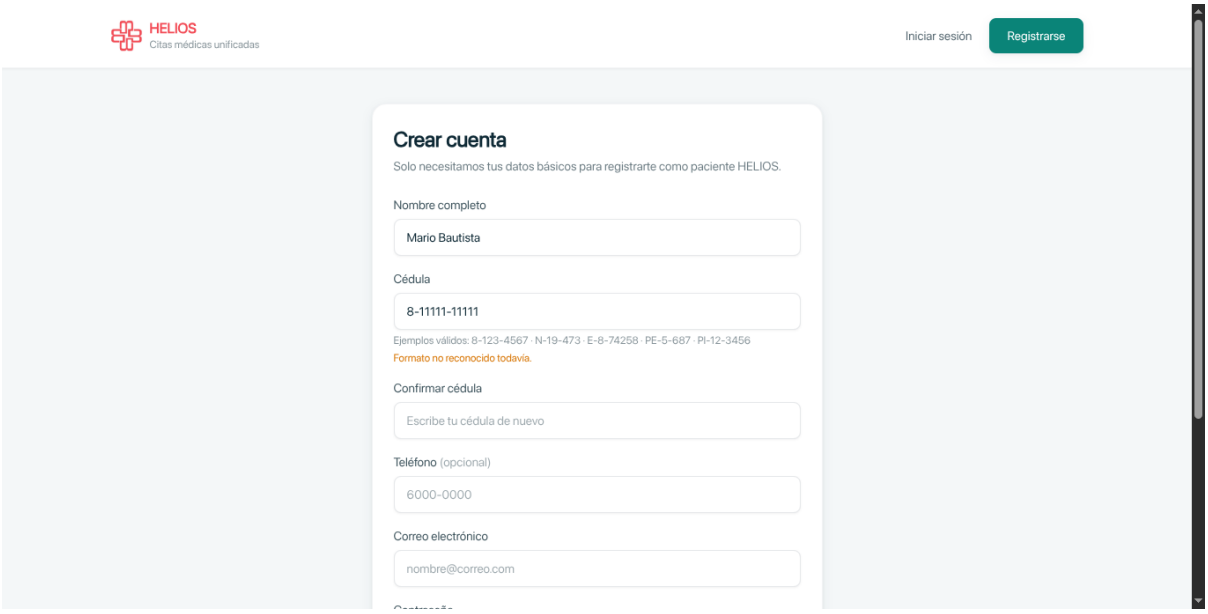


Evidencia 5 — Pantallas del sistema funcionando (demo)

1. Inicio de sesión:

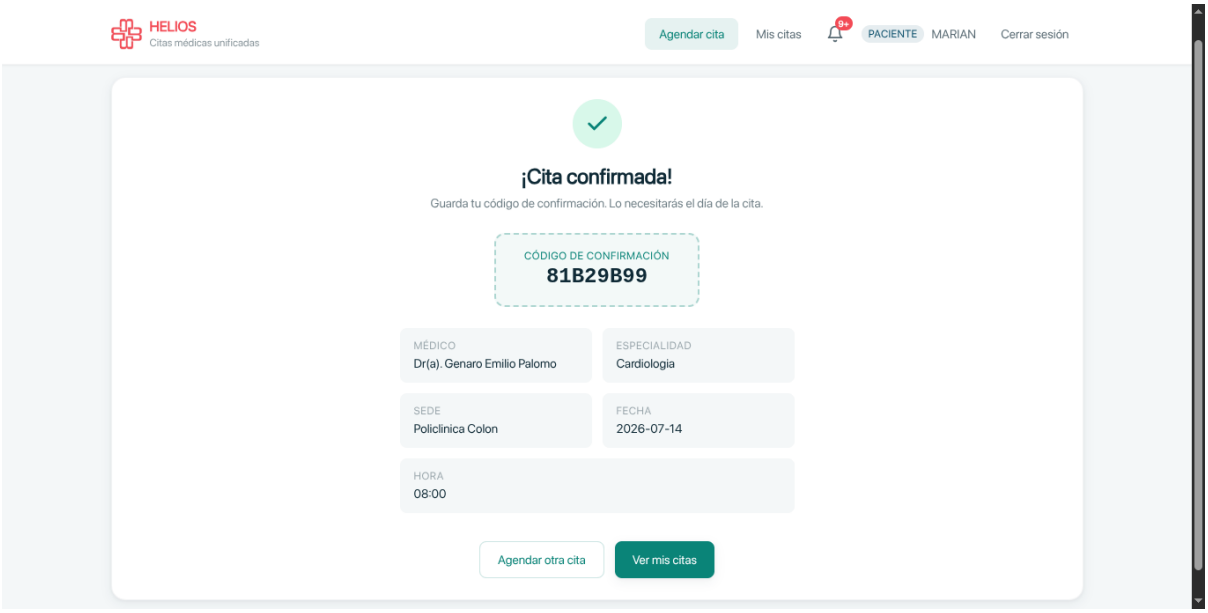


2. Formato de registro mostrando el error de cedula:

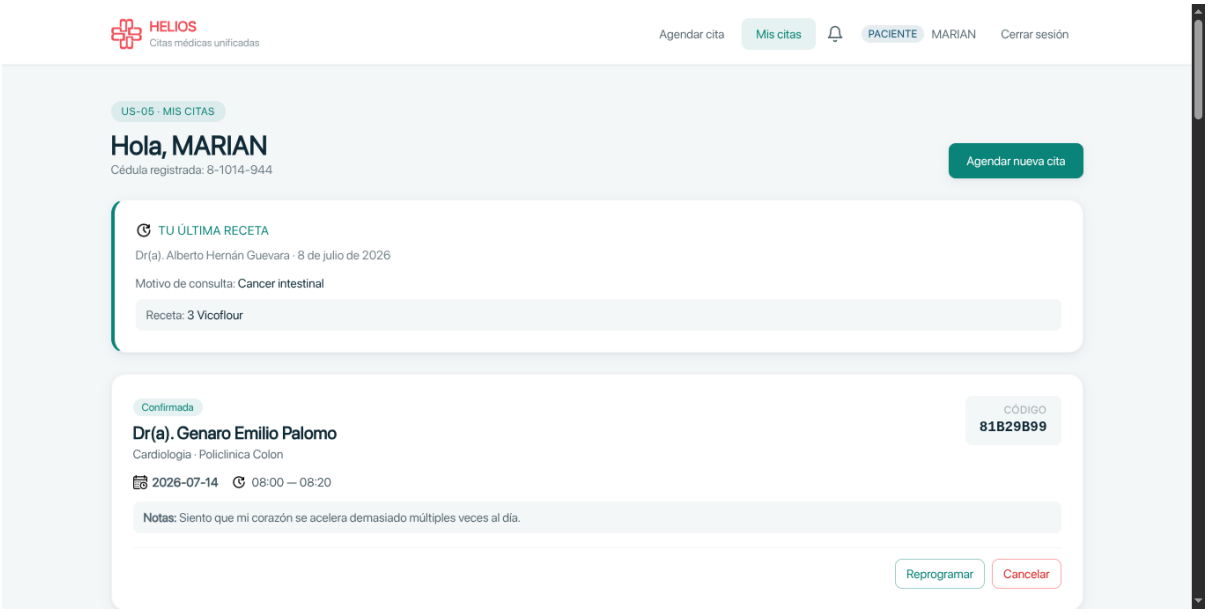


Evidencia 5 — Pantallas del sistema funcionando (demo)

3. Agendamiento de citas:



4. Pagina “Mis Citas”:



Evidencia 6 — Fragmento de código relevante

Nuevo regex de cedula aplicado:

```ts
export function isValidCedulaPanama(cedula: string): boolean {
  return CEDULA_REGEX.test(cedula)
}

/** Normaliza: quita espacios extremos y pasa a mayúsculas (N, E, PE). */
export function normalizeCedula(cedula: string): string {
  return cedula.trim().toUpperCase()
}

/**
 * Devuelve el tipo de una cédula válida, o 'invalida' si no cumple el formato.
 * Útil para etiquetar la cédula del usuario o ramificar lógica por tipo.
 */
export function tipoCedula(
  cedula: string,
): 'provincial' | 'naturalizado' | 'extranjero' | 'permiso-especial' | 'indigena' | 'invalida' {
  if (!isValidCedulaPanama(cedula)) return 'invalida'
  if (cedula.startsWith('PE-')) return 'permiso-especial'
  if (cedula.startsWith('PI-')) return 'indigena'
  if (cedula.startsWith('N-')) return 'naturalizado'
  if (cedula.startsWith('E-')) return 'extranjero'
  return 'provincial' // empieza con dígito de provincia
}

// ─────────────────────────────────────────────────────────────────────
// Helpers de UI (usados por Register.tsx). Se mantienen para no romper el form;
// se apoyan en el validador estricto de arriba.
// ─────────────────────────────────────────────────────────────────────

export type TipoCedula = 'natural' | 'n' | 'e' | 'pe' | 'pi' | 'desconocido'
```

Conclusiones

A continuación, se presentan las conclusiones individuales de cada integrante del equipo y la conclusión grupal sobre el proceso de pruebas y el producto final, seguidas de recomendaciones para una futura iteración.

Conclusiones Individuales

Jose Brown — Product Owner:

Como Product Owner aprendí que la calidad de software empieza mucho antes de las pruebas: definir criterios de aceptación claros y verificables para cada historia de usuario habría permitido detectar varios de los bugs en etapas más tempranas. El proceso de pruebas me demostró que el rol del PO también es validar que lo entregado cumple con lo prometido al usuario, no solo priorizar el backlog.

Ashley Martinez — QA Engineer:

Esta asignatura me hizo entender el valor real de un plan de pruebas estructurado bajo ISTQB. La auditoría de los 5 bugs me mostró que las técnicas formales (partición de equivalencia, valores límite, transición de estados) no son teoría: cada una me permitió reproducir y documentar defectos que a simple vista pasaban desapercibidos. Aprendí que un defecto bien documentado vale más que diez reportados a medias.

Ethan Gonzalez — Desarrollador Frontend:

Implementar las pruebas automatizadas con Vitest me cambió la forma de programar. Antes confiaba en probar a mano; ahora entiendo que cada hook y componente necesita su prueba. Corregir el BUG-05 (Realtime en useCitas) me enseñó que un requisito funcional no está completo hasta que tiene una prueba que lo respalde y prevenga regresiones.

Alejandro Saenz — Tech Lead / Desarrollador Backend:

Desde el backend y la base de datos, el proyecto me dejó claro que la integridad de datos no puede depender solo del frontend. El BUG-02 (race condition en registro) fue la lección más dura: las operaciones multi-tabla requieren transacciones o un manejo de rollback explícito. La seguridad con RLS de Supabase demostró ser una capa indispensable, no opcional.

Marian Barba — Scrum Master / DevOps:

Mantener el pipeline de CI/CD con SonarCloud y GitHub Actions me mostró que la calidad se automatiza o no se sostiene. La densidad de 5.88 bugs/KLOC antes de tener la suite completa fue una señal honesta de dónde estábamos. La retrospectiva MAD/SAD/GLAD me ayudó a sistematizar mejoras concretas en vez de quedarme en quejas.

Conclusión Grupal

Como equipo concluimos que HELIOS alcanzó los objetivos de calidad planteados: los 5 defectos detectados fueron corregidos y verificados mediante casos de prueba de regresión (Pass Rate = 100 %), la suite automatizada pasó a 138 pruebas en verde y el análisis estático no reportó vulnerabilidades críticas. El proceso nos demostró que aplicar estándares internacionales (ISTQB, ISO/IEC 25010) a un proyecto estudiantil no es burocracia: es lo que permite entregar un producto funcional, seguro y demostrable. La mayor lección grupal fue que la calidad se construye con disciplina desde el Sprint 0 (definir criterios, automatizar temprano, documentar defectos) y no se puede improvisar al final.

Recomendaciones para una Futura Iteración

• Ampliar la suite de pruebas de integración para cubrir todos los flujos críticos end-to-end, no solo los bugs corregidos.

• Incorporar un ambiente de staging separado de producción para ejecutar las pruebas sin riesgo sobre el sistema en uso.

• Implementar monitoreo 24/7 con alertas automáticas en Railway para detectar fallos del sistema de notificaciones en tiempo real.

• Agregar pruebas de carga y rendimiento con una herramienta como k6 para validar el comportamiento del sistema ante concurrencia.

• Aplicar el cuestionario SUS (System Usability Scale) a usuarios reales para medir la usabilidad de forma cuantitativa en cada módulo.

Referencias y Glosario

Referencias

[1] Equipo HELIOS, *Plan de Acción de la Retrospectiva MAD/SAD/GLAD – Sprint 5*, documento interno no publicado, 2026.

[2] Equipo HELIOS, *CLAUDE.md y Documentos de Decisiones de Arquitectura (ADR – Stack)*, documentación interna no publicada, 2026.

[3] Equipo HELIOS, *README del Proyecto HELIOS*, GitHub. Disponible: https://github.com/JahelDev23/helios-web. **[Accedido: 9-jul-2026].**

[4] Supabase Inc., *Supabase Documentation (Auth, Row Level Security, PostgreSQL, Realtime y Edge Functions)*. Disponible: https://supabase.com/docs. **[Accedido: 9-jul-2026].**

[5] Equipo HELIOS, *Historias de Usuario y Tablero Scrum (Jira – Tablero HELL)*, documentación interna no publicada, 2026.

[6] Equipo HELIOS, *Reporte de Bugs – Sprint 1 y Sprint 2 (Auditoría de Código basada en ISTQB)*, documento interno no publicado, 2026.

[7] Equipo HELIOS, *Casos de Prueba de Regresión de Bugs (TC-REG-01 a TC-REG-05)*, documento interno no publicado, 2026.

[8] International Software Testing Qualifications Board (ISTQB), *Foundation Level Syllabus v2018*. Disponible: https://istqb.org/certifications/certified-tester-foundation-level/. **[Accedido: 9-jul-2026].**

[9] International Organization for Standardization (ISO) e International Electrotechnical Commission (IEC), *ISO/IEC 25010:2011 — Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models*, Geneva, Switzerland, 2011.

[10] J. Brooke, "SUS: A 'Quick and Dirty' Usability Scale," en *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester y I. L. McClelland, Eds. London, U.K.: Taylor & Francis, 1996, pp. 189–194.

Glosario

Término / Sigla	Definición
RLS	Row Level Security: políticas de seguridad de PostgreSQL/Supabase que restringen el acceso a filas según el usuario autenticado.
CI/CD	Integración Continua / Despliegue Continuo. En HELIOS: GitHub Actions (CI) y Railway (CD).
Pass Rate	Porcentaje de casos de prueba exitosos sobre el total de casos ejecutados.
Sprint	Iteración de trabajo de una semana dentro del marco Scrum.
MAD/SAD/GLAD	Técnica de retrospectiva ágil (Mad = frustración, Sad = perjuicio, Glad = satisfacción).
Defecto bloqueante / crítico	Defecto que impide continuar con la ejecución de pruebas o el uso de una funcionalidad clave.
RPC	Remote Procedure Call: función PostgreSQL invocada desde el cliente (p. ej. crear_cita_atomica).
Realtime	Capacidad de Supabase para emitir eventos de cambios en la base de datos hacia los clientes suscritos.
KLOC	Kilo-Líneas de Código. Unidad usada para medir la densidad de defectos (bugs/KLOC).
Vitest	Framework de pruebas unitarias y de integración para proyectos basados en Vite.
RTL	React Testing Library: librería para probar componentes de React.